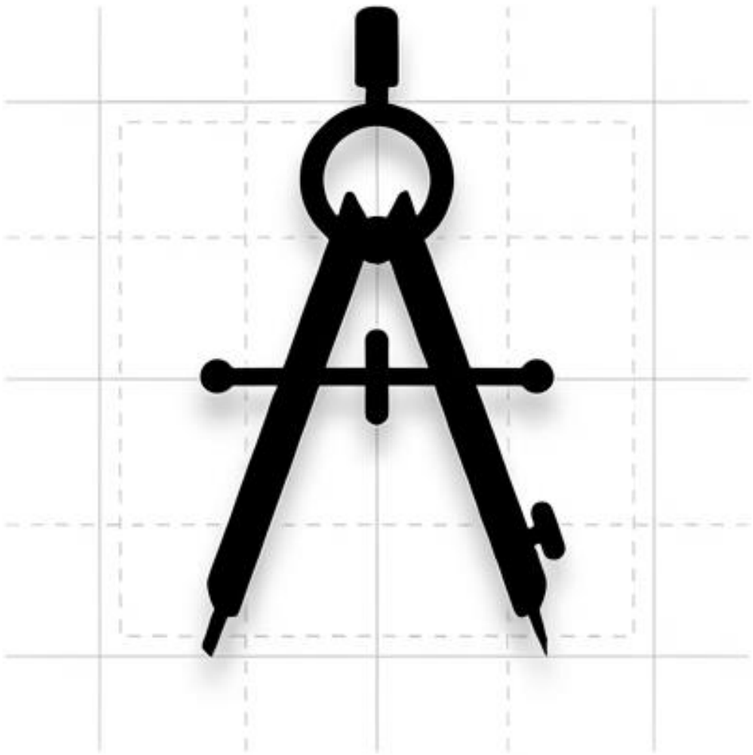
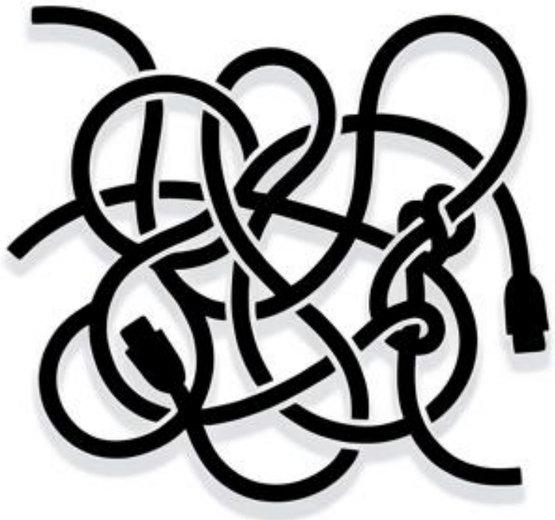


Object-Oriented Programming I

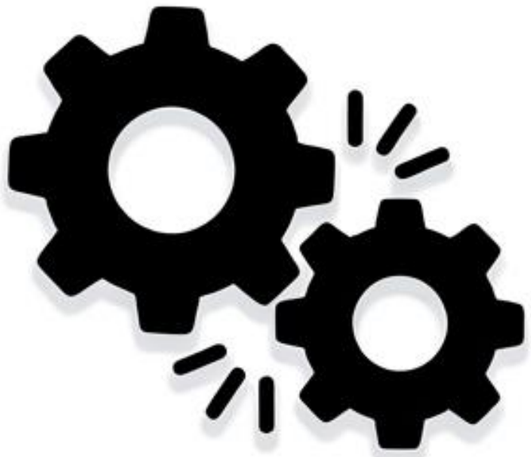
What is Object-Oriented Programming?

- A way of structuring software around “objects” instead of procedures
- Objects represent real-world or domain concepts
- Each object contains data (state) and behaviour (methods)
- Focuses on modelling problems before writing solutions
- Encourages structured, maintainable systems

Why Software Projects Fail



Growing complexity without structure

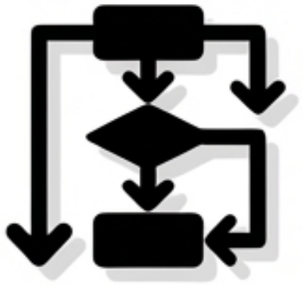


Mixing responsibilities

Lack of clear ownership of logic

Technology rarely fails first. Design does.

Procedural vs. Object Thinking



- Focus on steps and functions
- Data is passed around freely
- Logic often scattered

Procedural asks: What happens next?

- Focus on entities and responsibilities
- Data and behaviour grouped together
- Clear ownership of state

OOP asks: Who is responsible?

Defining the structure

Abstract Blueprint

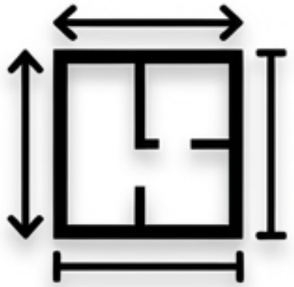
Encapsulating state

Name

Bedrooms

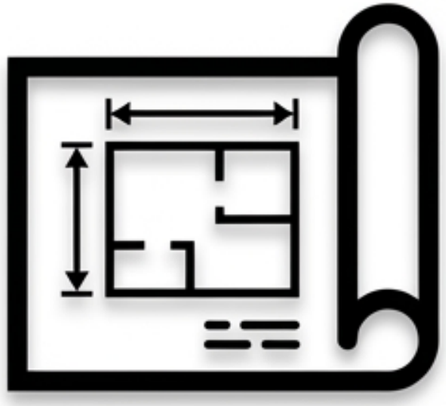
Doors

Windows



Area

The Blueprint vs. The Instance



The Class

Abstract definition of an entity.
Describes structure, not instances.

The Objects

Concrete instances. Each exists
independently with its own state.

Same structure. Different data.

Anatomy of an Object

Attributes (State)

Characteristics of an object. Stores internal values that can change over time.

Methods (Behaviours)

Defines what an object can do. Operates on the internal state.

The Constructor (__init__)

Runs upon creation. Defines how an object begins its life in a valid state.

Real-World Modeling: The House Class

Attributes
- name
- bedrooms
- doors
- windows
- area

Methods
- show_details()

Demonstrates encapsulation of data and behaviour.
Each house instance has independent values.

```python
class House:
    def __init__(self, name, bedroom, door, window, area):
        self.name = name
        self.bedroom = bedroom
        self.door = door
        self.window = window
        self.area = area

    def show_features(self):
        print("This is a", self.name)
        print("Bedroom: ", self.bedroom, "Doors: ", self.door,                    "Window: ", self.window, "Area: ",
self.area)

h1 = House("Barrington", 3, 2, 4, 120)
h2 = House("Kensington", 4, 6, 5, 300)

h1.show_features()
h2.show_features()
```

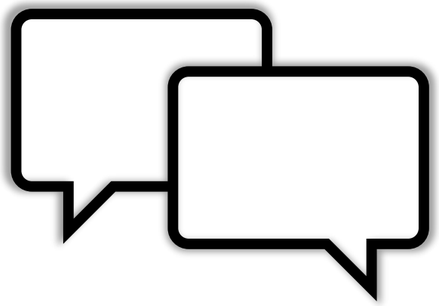
```python
class Animal:
    def __init__(self, name, age, species, height):
        self.name = name
        self.age = age
        self.species = species
        self.height = height

    def display_info(self):
        print(f"""”
        This is {self.name}, a {self.age}-year-old {self.species} that is {self.height} cm tall.
                    """)

a1 = Animal("Leo", 5, "Lion", 120)
a2 = Animal("Mia", 3, "Meerkat", 30)
a3 = Animal("Ella", 2, "Elephant", 250)

a1.display_info()
```

Let’s Catch Up



- A Class is defined once.
- x = Class(…) to create an object
- Notice self is not present in constructor call.

- h1 and h2 are separate objects that share the same Class design and type.
- x.methodName(…) to call a method for x

Team Management

The independent entity.

`Player`

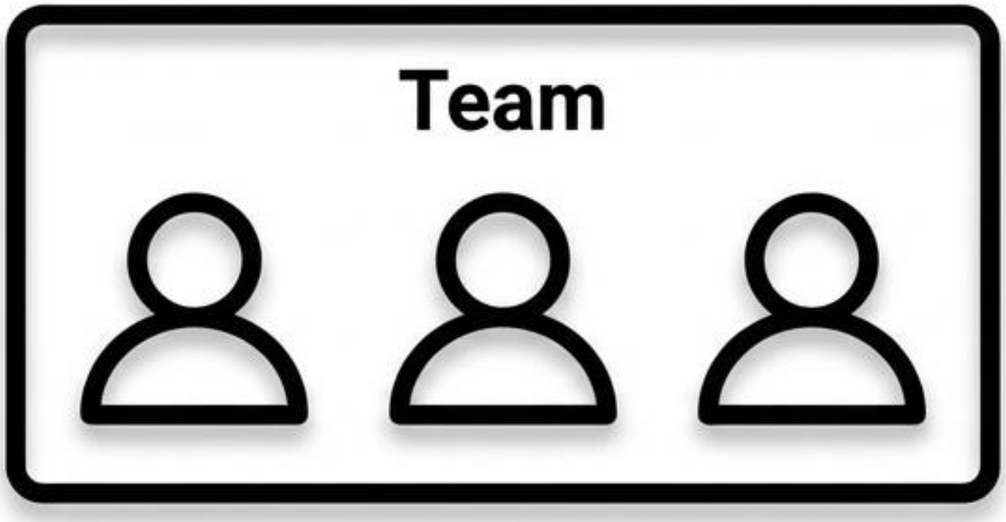
Maintains independent state.
Executes specific behavior.

The structural aggregator.

`Team`

Represents a collection.
Manages group dynamics.

Collaboration and the “Has-A” Relationship



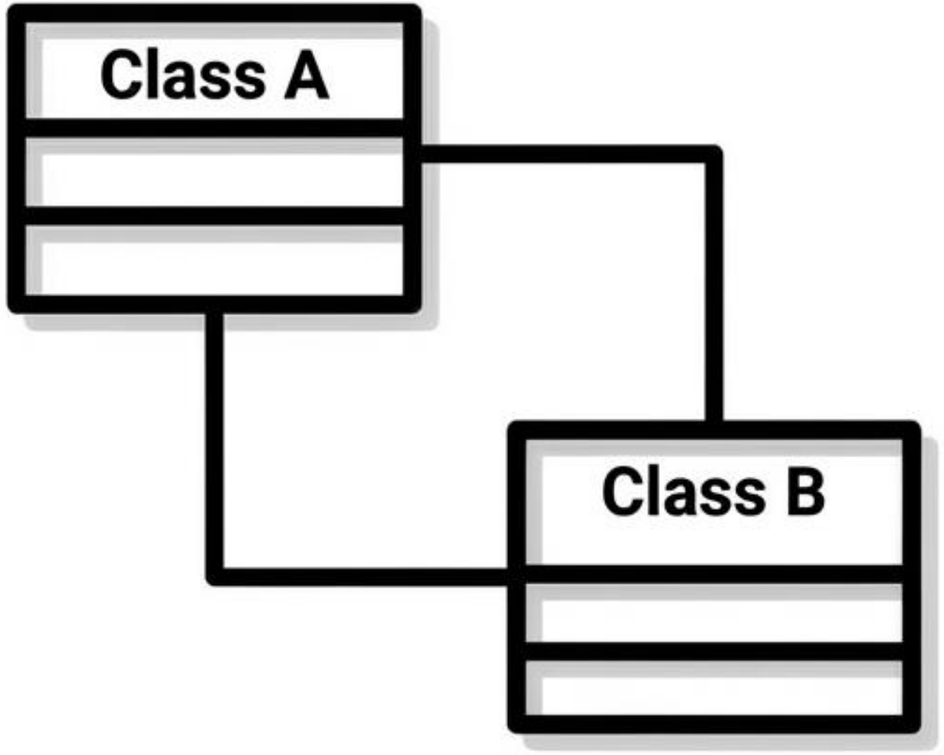
- Player: Represents an individual; manages its own data and behaviour.
- Team: Represents a collection; manages relationships between players.

Composition: **Models real-world ownership. A Team has Players.**

Single-class systems are rare. Complexity is managed through interaction.

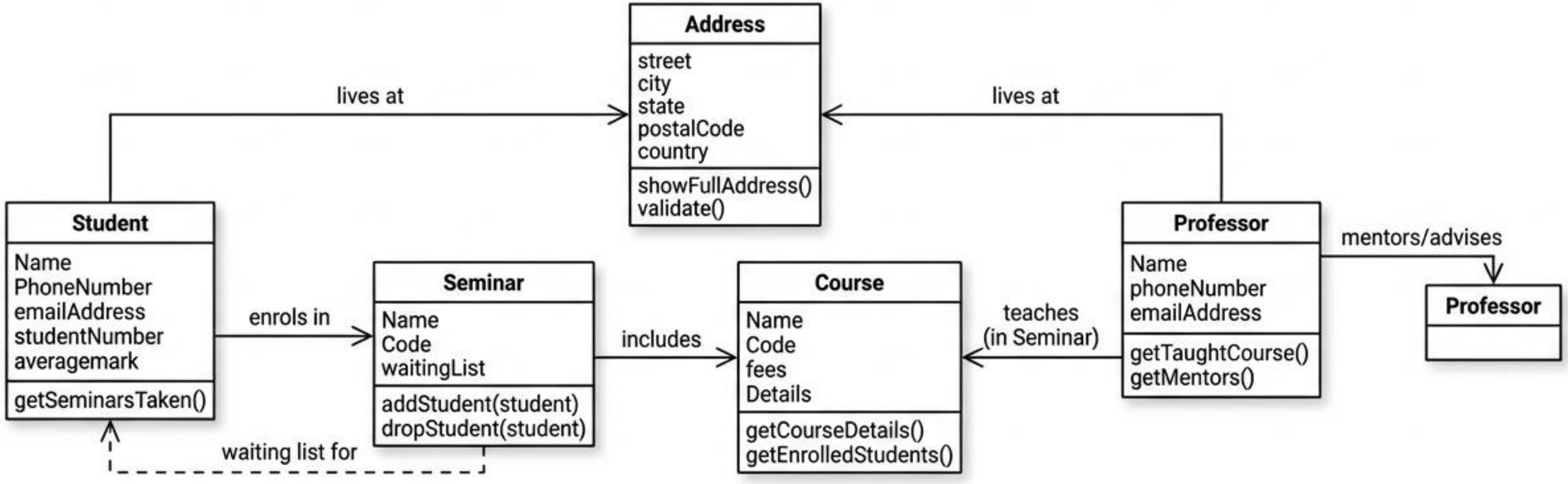
What is a Class Diagram?

- Visual representation of classes
- Shows attributes and methods
- Displays relationships between classes
- Used during planning and design
- Helps understand structure before coding



A class diagram is a blueprint for your code's structure.

Detailed Class Diagram Example



- Displays specific classes, attributes, and methods.
- Maps precise dependencies and relationships (e.g., enrolment, teaching, living).
- Illustrates concrete system structure before coding.

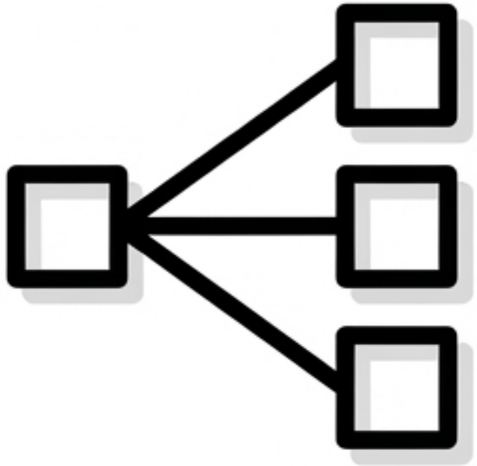
Design reduces chaos.

Mapping System Relationships

One-to-One

Example: Passport — Person

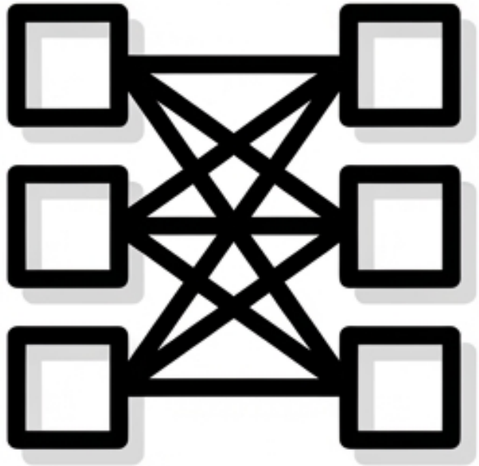
Simple dependency



One-to-Many

Example: Team — Players

Often implemented using lists



Many-to-Many

Example: Students — Courses

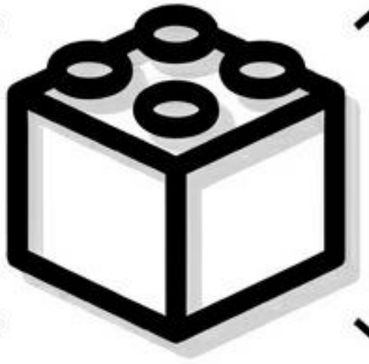
Requires intermediary structures

Complex relationships require careful design.

What Makes a Good Class?

Easy to modify
Adaptable to change.

Clear purpose
Focused, specific role.



Minimal dependencies
Low coupling with others.

Single responsibility
Handles only one function.

A class should simplify, not complicate.

Cohesion

Measures how related the responsibilities are within a class
High cohesion is desirable
All methods should relate to the same purpose
Low cohesion creates confusion and fragility.

Coupling

Measures dependency between classes
Low coupling is desirable
Changes in one class should not cascade widely
Loose connections improve flexibility.

Responsibility-Driven Design

Ask:
What is this class responsible for?
What does it know?
What does it do?
Assign responsibilities deliberately.
Do not let them drift.

What Modularisation Means: Modularisation means breaking a program into smaller, independent parts called modules. Each module:

has a clear responsibility

can be developed separately

can be reused

can be tested independently

Modularisation

Encapsulation

Hide internal state
Expose behaviour instead of raw data
Protect invariants
Prevent invalid states
Encapsulation safeguards system integrity.

Python’s Flexible Approach to OOP

“Why good design disciple matters in Python”

Python is less rigorous about OOP

Lack of public and private data can make systems harder to maintain

Languages like Java forces you to declare who can access your data

The Big Idea

In strict OOP languages, the language enforces rules.

In Python, the programmer enforces the rules

</> Java

```java
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}


account.balance = 1000000;
```

</> Python

```python
class BankAccount:
    def __init__(self):
        self.balance = 0

    def deposit(self, amount):
        self.balance += amount


account = BankAccount()

account.balance = 1000000

account.balance = -500
```

</> Python

```python
class BankAccount:
    def __init__(self):
        self.__balance = 0
```

_BankAccount__balance

A deliberate choice

Use OOP Intentionally

Python classes are optional

Key Takeaways

Model Reality

Map systems to real-world entities.

Define Structure

Classes create the blueprint.

Design Relational Intent

Connect objects purposefully.

Balance

High cohesion, low coupling.

Discipline

OOP is a framework for thinking.